



Institute of
Emerging
Technologies

**NAME: MUREED
ABBAS**

CLASS: BSCS V

SECTION: A

CMS ID : 023-24-0439

INS:Ma'am Muniza
Sub: Operating System

LAB :- FIVE

Task 1 — PCB and what gets copied on fork()

What is a PCB?

A Process Control Block (PCB) is the data structure the operating system kernel maintains for every process — essentially the process's administrative record. It typically holds:

- Process ID (PID) and parent process ID (PPID)
- Process state (running, ready, waiting/sleeping, stopped, zombie)
- Program counter (address of the next instruction to execute)
- CPU register values (saved on a context switch)
- CPU scheduling info (priority, nice value)
- Memory-management info (page tables, segment pointers, address-space limits)
- List of open files / file descriptor table
- Accounting info (CPU time used, time limits)

What gets copied when a child is created (fork())

When `fork()` creates a child, the kernel builds the child a brand-new PCB that starts as a near-identical copy of the parent's PCB: the register values/program counter (the child resumes at exactly the same point, right after the `fork()` call), the open file descriptor table (parent and child initially share the same open files), and the memory image (code, data, heap, stack — logically duplicated, though in practice most Linux kernels use copy-on-write to avoid physically copying pages until one side writes to them). What does NOT stay the same: the PID is new and unique to the child, the PPID in the child's PCB is set to the parent's PID, and accounting fields (CPU time used, etc.) start fresh at zero for the child.

Task 2 — One process prints odd numbers, the other prints even

Task

Create a new process using `fork()`. One process should print odd numbers, the other should print even numbers.

Design note

`fork()`'s return value is the only thing that lets a process tell which branch it is: zero means "I am the child", a positive value (the child's PID) means "I am the parent". The child prints the evens, the parent prints the odds, and the parent calls `wait(NULL)` so the child is properly reaped instead of being left as a zombie.

Solution — `tasks/task2_odd_even.c`

```
/* Task 2: Create a new process using fork(). One process prints odd
 * numbers, the other prints even numbers (1 to 10). */
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();
```

```

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        /* Child process: print even numbers */
        printf("Child (pid=%d, ppid=%d) printing EVEN numbers: ", getpid(), getppid());
        for (int i = 2; i <= 10; i += 2)
            printf("%d ", i);
        printf("\n");
    } else {
        /* Parent process: print odd numbers */
        printf("Parent (pid=%d) printing ODD numbers: ", getpid());
        for (int i = 1; i <= 10; i += 2)
            printf("%d ", i);
        printf("\n");
        wait(NULL); /* wait for child to avoid a zombie */
    }
    return 0;
}

```

Compile & run

```

user@ubuntu:~$ gcc task2_odd_even.c -o task2_odd_even
user@ubuntu:~$ ./task2_odd_even

```

Verified output

```

Child (pid=89, ppid=88) printing EVEN numbers: 2 4 6 8 10
Parent (pid=88) printing ODD numbers: 1 3 5 7 9

```

Because the parent and child are scheduled independently, the two lines can appear in either order on a different run — that is expected, normal fork() behaviour, not a bug.

Task 3 — Five processes, five unique ppid's

Task

Create five processes using fork(). Each process must print its own ppid, and every ppid printed must be unique (no repetition).

Design note

The obvious-looking approach — one parent calling fork() five times in a simple loop — fails the "no repetition" requirement: all five children would be created directly by the same original process, so they would all report the exact same ppid. The fix is to build a CHAIN instead of a star: process 1 forks process 2, process 2 forks process 3, and so on. Each process in the chain has a different process as its parent, so the five ppid's come out guaranteed distinct.

Solution — tasks/task3_five_unique_ppid.c

```

/* Task 3: Create five processes using fork(). Each process prints its own
 * ppid, and every ppid printed must be unique (no repetition).
 *
 * Design: if all 5 children were created directly by one parent (a plain
 * loop calling fork() 5 times from the SAME process), every child would

```

```

* share the very same ppid -- the original parent's pid -- which breaks
* the "no repetition" requirement. Instead we build a CHAIN: process 1
* forks process 2, process 2 forks process 3, and so on. Each process's
* parent is a different process in the chain, so all 5 ppids are
* guaranteed to be distinct. */
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

void spawnChain(int remaining, int level) {
    if (remaining == 0) return;

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return;
    } else if (pid == 0) {
        /* child: this is the next process in the chain */
        printf("Process #%d -> pid=%d, ppid=%d\n", level, getpid(), getppid());
        fflush(stdout); /* _exit() skips the normal stdio flush */
        spawnChain(remaining - 1, level + 1);
        _exit(0);
    } else {
        /* this process's job is done once its one child finishes */
        wait(NULL);
    }
}

int main() {
    spawnChain(5, 1);
    return 0;
}

```

Compile & run

```

user@ubuntu:~$ gcc task3_five_unique_ppid.c -o task3_five_unique_ppid
user@ubuntu:~$ ./task3_five_unique_ppid

```

Verified output

```

Process #1 -> pid=112, ppid=111
Process #2 -> pid=113, ppid=112
Process #3 -> pid=114, ppid=113
Process #4 -> pid=115, ppid=114
Process #5 -> pid=116, ppid=115

```

All five ppids (111, 112, 113, 114, 115) are different, exactly as required.

Task 4 — Parent waits specifically for the second child

Task

Create three processes using `fork()`. The parent process must wait specifically for the second child to terminate (not just "any" child).

Design note

Plain `wait(NULL)` blocks until ANY child terminates — whichever one happens to finish first — which cannot guarantee you are waiting on child #2 specifically. `waitpid(pid, &status, 0)` takes a target PID and blocks only until that exact process terminates, which is exactly the tool needed here. To prove the behaviour is really pid-specific rather than coincidental, child 2 is given the shortest sleep (1s) and child 3 the longest (5s): the parent's `waitpid(child2, ...)` call returns as soon as child 2's 1 second is up, well before child 1 (3s) or child 3 (5s) have finished, and only afterwards does the parent reap the remaining two with ordinary `waitpid()` calls so no zombies are left behind.

Solution — `tasks/task4_wait_second_child.c`

```
/* Task 4: Create three processes using fork(). The parent must wait
 * specifically for the SECOND child to terminate (not just any child). */
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t child1, child2, child3;

    child1 = fork();
    if (child1 == 0) {
        printf("Child 1: pid=%d, ppid=%d -- sleeping 3s\n", getpid(), getppid());
        fflush(stdout);
        sleep(3);
        printf("Child 1: done\n");
        fflush(stdout);
        _exit(0);
    }

    child2 = fork();
    if (child2 == 0) {
        printf("Child 2: pid=%d, ppid=%d -- sleeping 1s\n", getpid(), getppid());
        fflush(stdout);
        sleep(1);
        printf("Child 2: done\n");
        fflush(stdout);
        _exit(0);
    }

    child3 = fork();
    if (child3 == 0) {
        printf("Child 3: pid=%d, ppid=%d -- sleeping 5s\n", getpid(), getppid());
        fflush(stdout);
        sleep(5);
        printf("Child 3: done\n");
        fflush(stdout);
        _exit(0);
    }

    /* Parent: wait specifically for child2 using waitpid(), regardless of
     * the order in which child1 / child2 / child3 actually finish. */
    printf("Parent: pid=%d -- created children %d, %d, %d\n", getpid(), child1, child2,
    child3);
    printf("Parent: waiting specifically for child 2 (pid=%d)...\n", child2);
    fflush(stdout);
```

```

waitpid(child2, NULL, 0);
printf("Parent: child 2 (pid=%d) has terminated -- parent continues now\n", child2);
fflush(stdout);

/* Reap the other two children so they don't become zombies */
waitpid(child1, NULL, 0);
waitpid(child3, NULL, 0);
printf("Parent: all children finished, exiting\n");
return 0;
}

```

Compile & run

```

user@ubuntu:~$ gcc task4_wait_second_child.c -o task4_wait_second_child
user@ubuntu:~$ ./task4_wait_second_child

```

Verified output

```

Child 1: pid=136, ppid=135 -- sleeping 3s
Parent: pid=135 -- created children 136, 137, 138
Parent: waiting specifically for child 2 (pid=137)...
Child 2: pid=137, ppid=135 -- sleeping 1s
Child 3: pid=138, ppid=135 -- sleeping 5s
Child 2: done
Parent: child 2 (pid=137) has terminated -- parent continues now
Child 1: done
Child 3: done
Parent: all children finished, exiting

```

"Parent: child 2 has terminated" prints right after "Child 2: done" — about 1 second in — and well before "Child 1: done" / "Child 3: done" appear roughly 2 and 4 seconds later. The parent genuinely resumed the moment child 2 (and only child 2) finished.

Task 5 — Command to prioritize a process

Task

Give the command used to prioritize a process, with an example.

Answer

Linux schedules processes partly based on a "niceness" value from -20 (highest priority / least nice to others) to +19 (lowest priority / most nice to others). Two commands control it:

- `nice` — launches a brand-new process with a chosen niceness (default niceness for a normal launch is 0).
- `renice` — changes the niceness of a process that is already running, given its PID.

Example 1 — launch a process with a custom priority using nice

```

user@ubuntu:~$ nice -n 10 sleep 5 &
user@ubuntu:~$ ps -o pid,ni,cmd -p 166

```

Verified output

```
PID  NI  CMD
166  10  sleep 5
```

Example 2 — change the priority of an already-running process using renice

```
user@ubuntu:~$ renice -n -5 -p 166
user@ubuntu:~$ ps -o pid,ni,cmd -p 166
```

Verified output

```
166 (process ID) old priority 10, new priority -5
PID  NI  CMD
166  -5  sleep 5
```

Note: lowering the niceness below 0 (making a process higher priority) requires root/superuser privileges; an ordinary user can only move a process's niceness upward (make it nicer / lower priority).

Task 6 — Checking free disk space and RAM

Task

Give the command used to check free space on the hard disk and on RAM.

Answer

- df ("disk free") reports free/used space per mounted filesystem. The -h flag prints sizes in human-readable units (GB/MB) instead of raw blocks.
- free reports total/used/free physical RAM and swap. Again, -h gives human-readable units.

Example — df -h

```
user@ubuntu:~$ df -h
```

Verified output

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	4.0G	4.0K	4.0G	1%	/dev/shm
/dev/vda	252G	8.6G	10G	47%	/
tmpfs	2.0G	0	2.0G	0%	/sys/fs/cgroup
tmpfs	1.0M	0	1.0M	0%	/run/sandbox-telemetry

Example — free -h

```
user@ubuntu:~$ free -h
```

Verified output

	total	used	free	shared	buff/cache	available
Mem:	3.9Gi	217Mi	3.8Gi	13Mi	110Mi	3.7Gi
Swap:	0B	0B	0B			

Task 7 — How many processes does fork(); fork(); fork(); create?

Task

void main() { fork(); fork(); fork(); } — how many processes get created?

Reasoning

Each `fork()` call doubles the number of processes currently alive, because every existing process (the original plus every process created by earlier `fork()` calls) independently executes each subsequent `fork()` statement and splits into two. Starting from 1 process: after the first `fork()` there are 2; after the second `fork()` each of those 2 splits, giving 4; after the third `fork()` each of those 4 splits, giving 8. In general, n consecutive `fork()` calls (with no conditional branching) produce 2^n total processes. Here $n = 3$, so $2^3 = 8$ processes in total — the 1 original process plus 7 newly created child processes.

Empirical proof

To confirm this isn't just theoretical, the equivalent program below has every surviving process append one byte to a shared log file right after the three `fork()` calls; counting the bytes in the log file afterwards counts exactly how many processes reached that line.

Solution — `tasks/task7_fork_count.c`

```
/* Task 7: How many processes are created by the following code?
 *      void main() { fork(); fork(); fork(); }
 *
 * This program instruments each fork() call with a counter write to a
 * shared file so we can empirically COUNT how many processes actually
 * run, instead of just reasoning about it on paper. */
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/file.h>
#include <fcntl.h>

void count_process(const char *path) {
    int fd = open(path, O_WRONLY | O_APPEND | O_CREAT, 0644);
    flock(fd, LOCK_EX);
    write(fd, "x", 1);
    flock(fd, LOCK_UN);
    close(fd);
}

int main() {
    const char *logfile = "process_count.log";
    remove(logfile);

    fork();
    fork();
    fork();

    /* every one of the resulting processes reaches this point and logs
     * itself exactly once */
    count_process(logfile);

    return 0;
}
```

Compile & run

```
user@ubuntu:~$ gcc task7_fork_count.c -o task7_fork_count
user@ubuntu:~$ ./task7_fork_count
```

```
user@ubuntu:~$ wc -c process_count.log
```

Verified output

```
8 process_count.log  
Total processes that executed the final line: 8
```

Confirmed: exactly 8 processes executed past the three fork() calls, matching the $2^3 = 8$ prediction.

How to Use These Files

The four C programs (Tasks 2, 3, 4 and 7) are provided separately in OS_Lab5_Solutions_Source.zip, inside a tasks/ folder. Each was compiled and run with gcc 13.3.0 on Ubuntu 24.04.

General recipe

```
user@ubuntu:~$ gcc <file>.c -o <program>
user@ubuntu:~$ ./<program>
```

Tasks 1, 5 and 6 are conceptual/command questions and have no accompanying source file — their commands (nice, renice, df, free) are already installed on any standard Ubuntu system and can be run directly as shown above.